

Making Maps and Cartographic Design

Alexander Abuabara

DAEN 489 ISEN 489 Spring 2026



ENGINEERING
TEXAS A&M UNIVERSITY

Project Proposal *Updates*

- Spreadsheet to help organize the groups

https://docs.google.com/spreadsheets/d/1RqVF_3GTnQsld3ikscjSPqKV1eLFaDH0ql4oAXluFZs/edit?usp=sharing

- One idea for a project:
 - Determine in multiple years if there is a statistically significant difference (*paired sample t-test*) between the aggregated LandScan population estimates and the Census residential population (where people live) at multiple census geometric boundary levels (tract, county, zip-code, etc.)

Introduction

- Cartography is the intersection of **art**, **science**, and **technology**
 - *Mapmaking*
 - Gerardus Mercator (1512–1594) is generally considered the most famous cartographer in history.
 - A **Flemish** geographer, he revolutionized navigation by developing the [Mercator projection](#) in 1569, a cylindrical map projection still widely used today.
 - He also popularized the term "atlas" for a collection of maps.
- **Goal:** To communicate complex spatial information clearly and accurately to a specific audience.
- **Key:** What is the story you want your map to tell?
 - Design is not just "making it pretty" but functional.
 - Poorly designed maps can leave a bad impression.
 - Common design issues include:
 - Poor placement, size and readability of text,
 - Careless selection of colors.

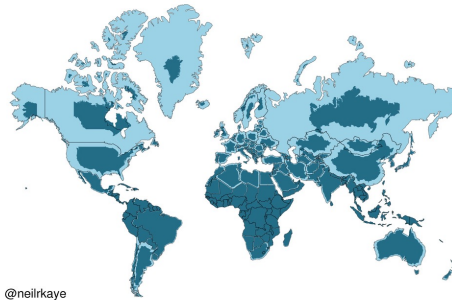


https://www.reddit.com/r/GISmemes/comments/kqax4y/anyone_with_me/

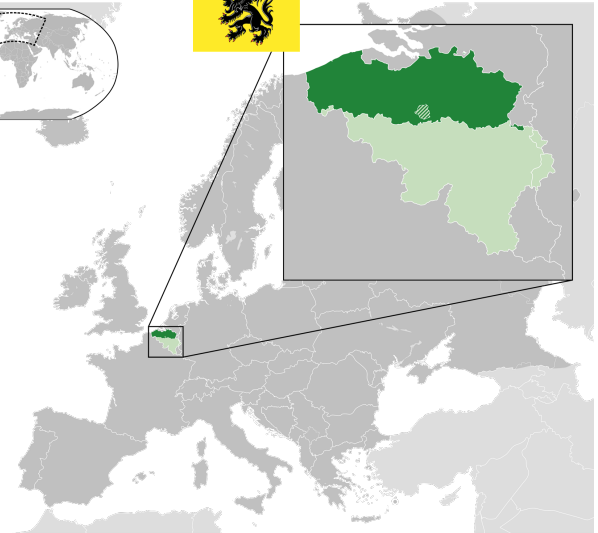
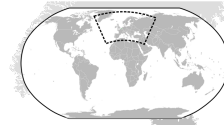
Flemish people or Flemings



World Mercator projection with true country size added



@neilrkaye

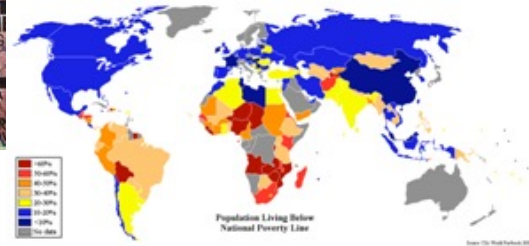
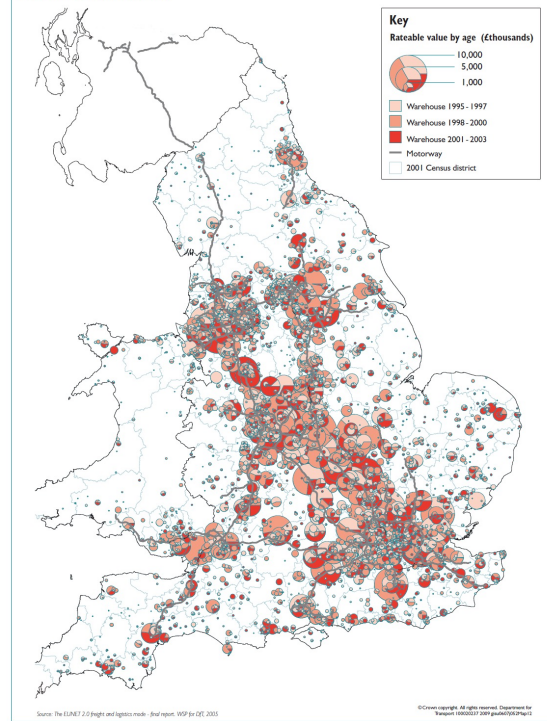


https://en.wikipedia.org/wiki/Flemish_people

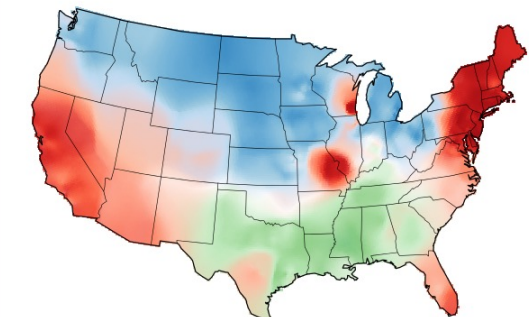


2.1 HOW WE TRAVEL: THE PRESSURES ON THE UK TRANSPORT NETWORK

Figure 1.I7: Location of new warehouses with concentrations at motorway intersections, 1995-2003

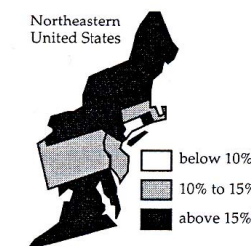


What is your generic term for a sweetened carbonated beverage?

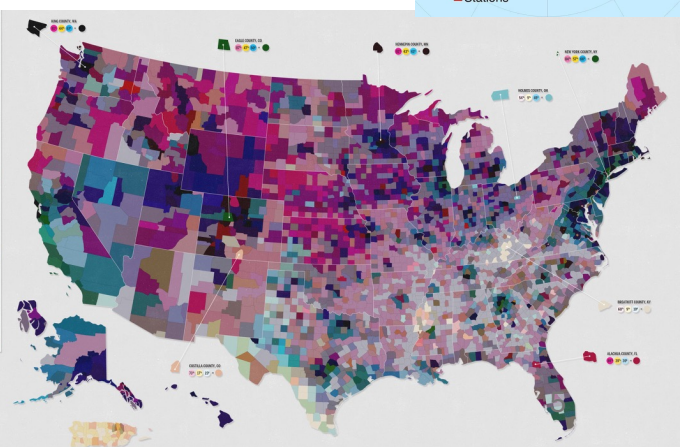
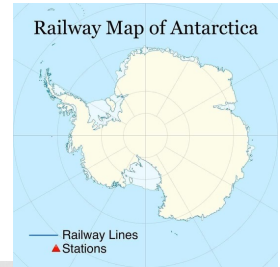
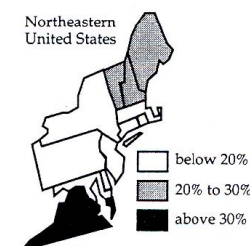


- soda
- pop
- coke
- soft drink

Occupied Housing Units Lacking a Telephone, 1960



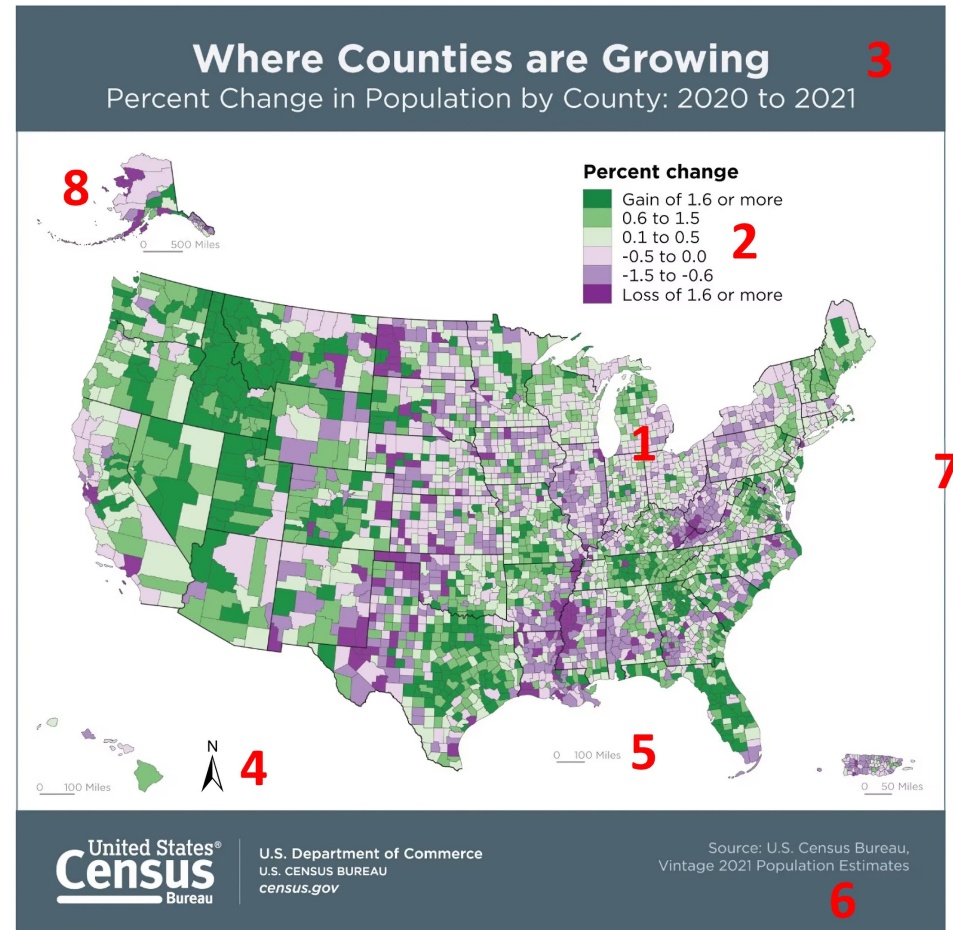
Occupied Housing Units Lacking a Telephone, 1960



Static maps

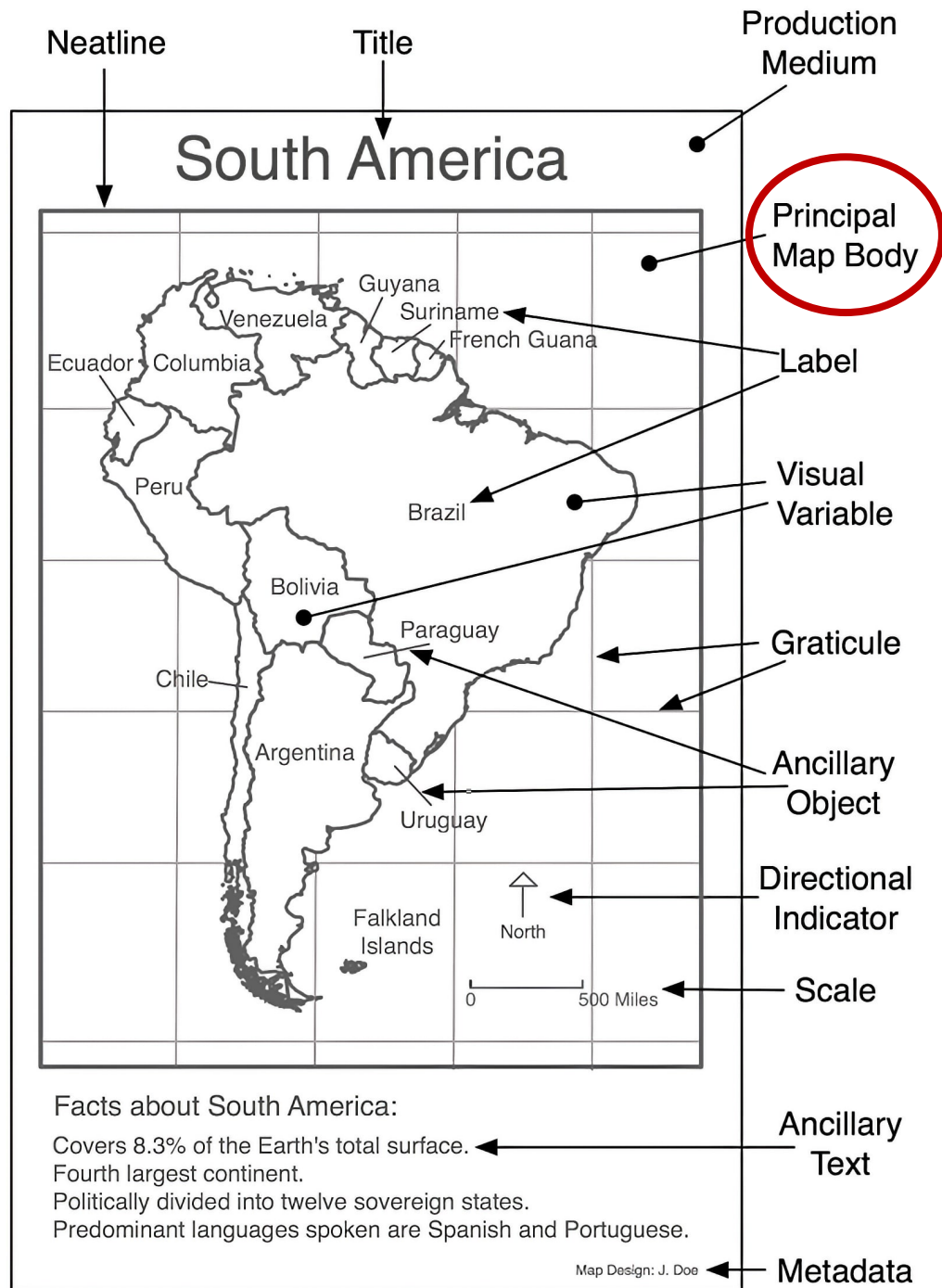
Map showing the elements that can be part of a map layout

1. Data (or map) frame
2. Map legend
3. Map title
4. North arrow
5. Map scale bar
6. Metadata (or map citation)
7. Border
8. Inset (or locator) map

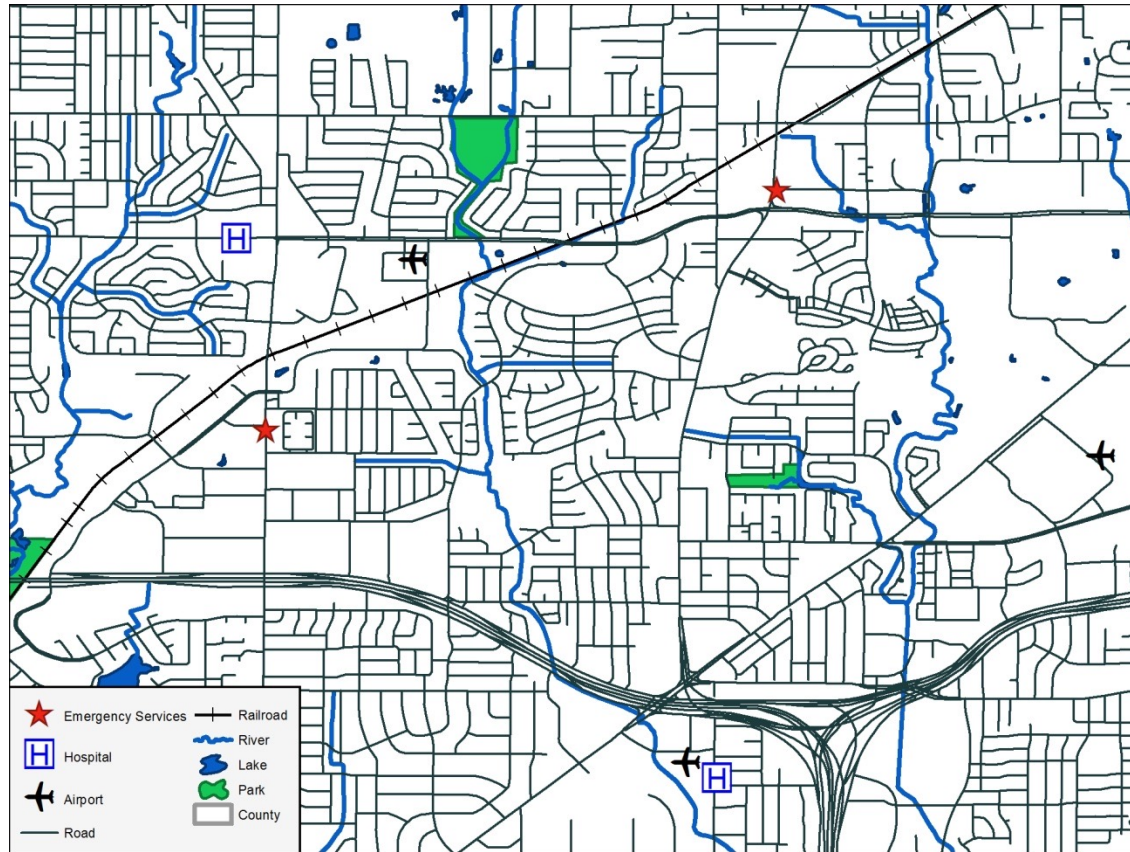




The map body map element is the **main focus** of the map and should be visually dominant in every case. The map body contains the features important to the message of the map. The map body should be made as large as possible on the medium but should also leave room for other map elements and should not crowd the edges so that the map has a little "breathing room". The map body is the only map element that is absolutely required when composing a map. However, the map is often much more useful when other map elements are included.



A General Reference Map



This map legend has two columns of information that are roughly equal in size. Each map feature is identified with a symbol that is identical to the symbol found on the map followed by the name of the map feature to the right of the symbol. On this general reference map legend common features such as lakes, parks, rivers, railroads, and roads are included, where they may not be included on thematic maps.

Basic Steps to Create a Basemap

- Install/load Libraries
 - **tigris** package to fetch the data
 - **ggplot2** (or **tmap**) for visualization.
- Enable Caching
 - Use **options(tigris_use_cache = TRUE)** to save downloaded files locally, preventing repeated downloads.
- Fetch Geographic Data
 - Call functions like **states()**, **counties()**, or **roads()** to retrieve the desired geometry.
- Plot the Data
 - Use **geom_sf()** in ggplot2 to render the spatial object.

Example 1

```
library(tigris)
library(ggplot2)

# Cache data for faster future use (recommended)
options(tigris_use_cache = TRUE)

# 1. Fetch U.S. state boundaries
# cb = TRUE uses a lower-resolution "Cartographic Boundary" for faster plotting
us_states <- states(cb = TRUE, resolution = "20m")

# 2. Shift Alaska, Hawaii, and Puerto Rico for a standard "national" view
us_states_shifted <- shift_geometry(us_states)

# 3. Create the basemap
ggplot(data = us_states_shifted) +
  geom_sf(fill = "whitesmoke", color = "darkgray") +
  theme_void() +
  labs(title = "U.S. State Basemap")
```


Example 2

```
library(tigris)
library(ggplot2)
library(sf)
library(dplyr)

options(tigris_use_cache = TRUE)

# 1. Get County Boundary
county_bound <- counties(state = "TX", cb = TRUE) %>%
  filter(NAME == "Travis")

# 2. Get Water Bodies
county_water <- area_water(state = "TX", county = "Travis")

# 3. Get Roads (Primary and Secondary for clarity; 'roads()' gets every street)
county_roads <- primary_secondary_roads(state = "TX") %>%
  st_intersection(county_bound) # Clip roads to the county boundary
```

```

ggplot() +
  # Layer 1: County Background
  geom_sf(data = county_bound, fill = "#fdfdfd", color = "black", size = 0.5) +

  # Layer 2: Water
  geom_sf(data = county_water, fill = "#a2d2ff", color = "transparent") +

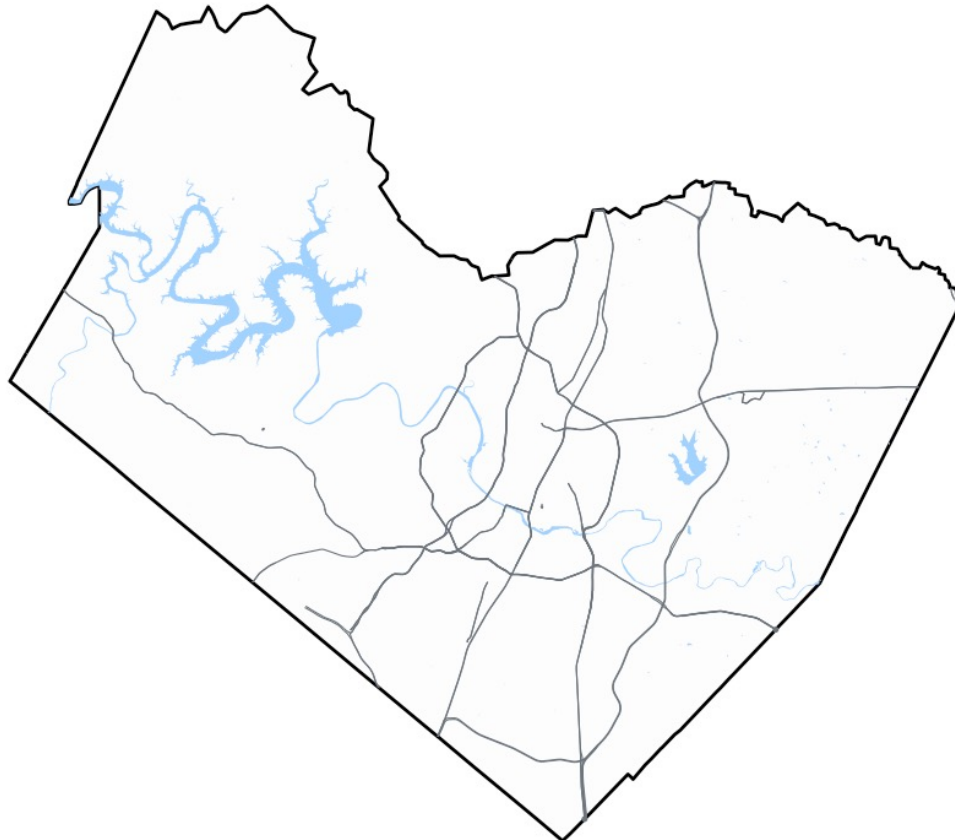
  # Layer 3: Roads
  geom_sf(data = county_roads, color = "#6c757d", size = 0.2) +

  # Formatting
  theme_void() +
  labs(
    title = "Basemap of Travis County, TX",
    subtitle = "Including Waterways and Primary/Secondary Roads",
    caption = "Source: US Census Bureau via tigris"
  ) +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5),
    panel.background = element_rect(fill = "white", color = NA)
  )

```

Basemap of Travis County, TX

Including Waterways and Primary/Secondary Roads



Source: US Census Bureau

```

library(ggspatial)

ggplot() +
  geom_sf(data = county_bound, fill = "wheat", color = "black", size = 0.5) +
  geom_sf(data = county_water, fill = "#a2d2ff", color = "transparent") +
  geom_sf(data = county_roads, color = "#6c757d", size = 0.2) +

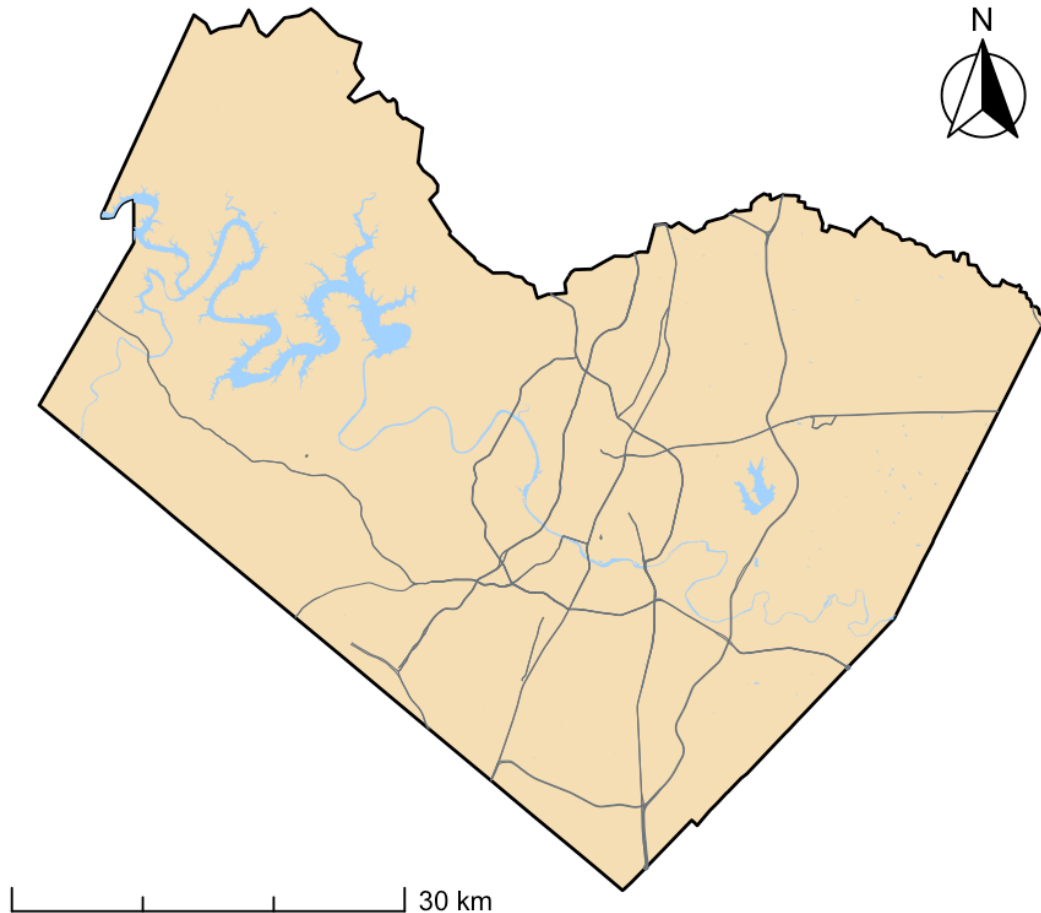
  # Add Scale Bar
  annotation_scale(
    location = "bl",          # 'bl' = bottom left
    width_hint = 0.4,        # Proportion of the map the scale bar should take
    style = "ticks"          # Can also use "bar"
  ) +

  # Add North Arrow
  annotation_north_arrow(
    location = "tr",          # 'tr' = top right
    which_north = "true",     # Aligns to true north
    pad_x = unit(0.2, "in"),
    pad_y = unit(0.2, "in"),
    style = north_arrow_fancy_orienteering()
  ) +

  theme_void() +
  labs(
    title = "Basemap of Travis County, TX",
    caption = "Caption: Scale and orientation provided by ggspatial"
  )

```


Basemap of Travis County, TX



Caption: Scale and orientation provided by ggspatial

```

# Filter for large water bodies (in square meters)
large_water <- county_water %>%
  mutate(area = as.numeric(st_area(.))) %>%
  filter(area > 1000000) # Adjust this number based on your specific county

# Get a cleaned list of unique primary roads for labeling
road_labels <- county_roads %>%
  filter(!is.na(FULLNAME)) %>%
  group_by(FULLNAME) %>%
  summarize() %>%      # This dissolves multi-line geometries for cleaner labeling
  st_centroid()         # Place text at the center point of the road

library(ggrepel)        # Optional but much better for label placement

ggplot() +
  geom_sf(data = county_bound, fill = "#fdfdfd", color = "black") +
  geom_sf(data = county_water, fill = "#a2d2ff", color = "transparent") +
  geom_sf(data = county_roads, color = "#6c757d", size = 0.2) +

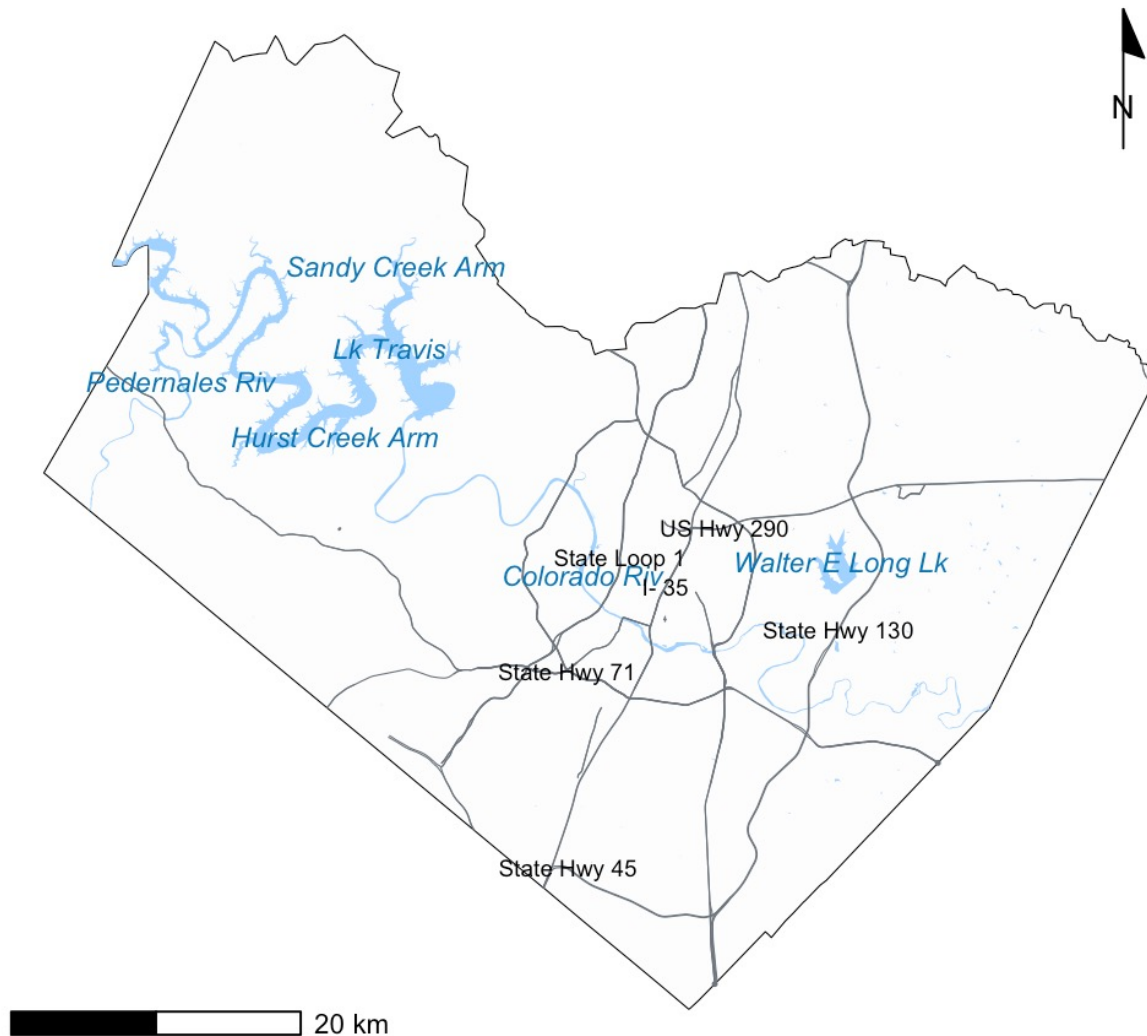
  # Label Water Bodies
  geom_sf_text(data = large_water, aes(label = FULLNAME),
    color = "#0077b6", fontface = "italic", size = 3, check_overlap = TRUE) +
  # Label Roads
  geom_sf_text(data = road_labels, aes(label = FULLNAME),
    color = "black", size = 2.5, check_overlap = TRUE) +

  annotation_scale(location = "bl") +
  annotation_north_arrow(location = "tr", style = north_arrow_minimal()) +
  theme_void()

```

```
# A cleaned list of unique primary roads for labeling
road_labels <- county_roads %>%
  filter(!is.na(FULLNAME)) %>%
  group_by(FULLNAME) %>%
  summarize() %>%      # This dissolves multi-line geometries for cleaner labeling
  st_centroid() %>%    # Place text at the center point of the road
  ungroup() %>%
  slice_sample(n = 10) # Randomly pick 10
```

```
# Or longest roads
road_labels <- county_roads %>%
  filter(!is.na(FULLNAME)) %>%
  group_by(FULLNAME) %>%
  summarize() %>%      # Merges all segments of the same road name
  mutate(road_length = st_length(geometry)) %>%
  arrange(desc(road_length)) %>% # Sort by length
  slice_head(n = 7) %>%      # Take the top 7
  st_point_on_surface()      # Better than centroid for winding roads
```



Data Representation

Thematic Maps

- A popular way of presenting information about spatial patterns is to use a thematic map.
- **Data is aggregated** by predefined spatial units (geo-administrative layers, e.g., countries, states, or districts).
- A broader category that includes various types of maps (e.g., choropleth, dot density, proportional symbols, isopleth maps, etc.)
- A thematic map shows the spatial distribution of one or more specific data themes for selected geographic areas.
- The data may be **quantitative** (e.g., percentage population change, income distribution) or **qualitative** (e.g., predominant soil types).
- The basic way to create a vector thematic map is to identify the field in the attribute table of the map layer that will be used for mapping.
- Note, the basemaps can be used for **context**!

Data Representation

Thematic Maps

- **Vector data** have **attributes** that describe **properties of the features**.
 - The attributes are stored in a **table**.
 - **Rows** are records.
 - There is **one record per feature** in the vector layer.
 - **Columns** are **fields**.
 - Fields represent **properties** of the feature (e.g. height, color etc.)
 - Fields can contain **numerical**, **string** (text) and **date** information.
 - The **attribute data** can be used to determine how it is **symbolized**.
 - If the attribute of a vector layer **is not used** to determine its symbology, it is drawn using a **single symbol only**.
 - **Graduated color** symbology groups the data into **discrete classes**.
 - **Continuous color** symbology assigns colors from a color range to the features based on their attributes.
- **Classifying** attribute data (into nominal, ordinal, interval, and ratio types) helps in selecting appropriate spatial analysis and thematic mapping techniques.
- **Normalized data*** (e.g., percentages or densities) instead of raw count values in a choropleth map helps to prevent misleading interpretations due to differences in area size.

* **normalization** (min-max scaling to a range), **standardization** (type of normalization, but using z-score/mean centering)

tmap basics

- Like **ggplot2**, **tmap** is based on the idea of a 'grammar of graphics'.
- This involves a separation between the input data and the aesthetics (how data are visualized):
 - each input dataset can be 'mapped' in a range of different ways including location on the map (defined by data's geometry), color, and other visual variables.
- The basic building block is **tm_shape()** (which defines input data: a **vector** or **raster** object), followed by one or more-layer elements such as **tm_fill()** and **tm_dots()**.

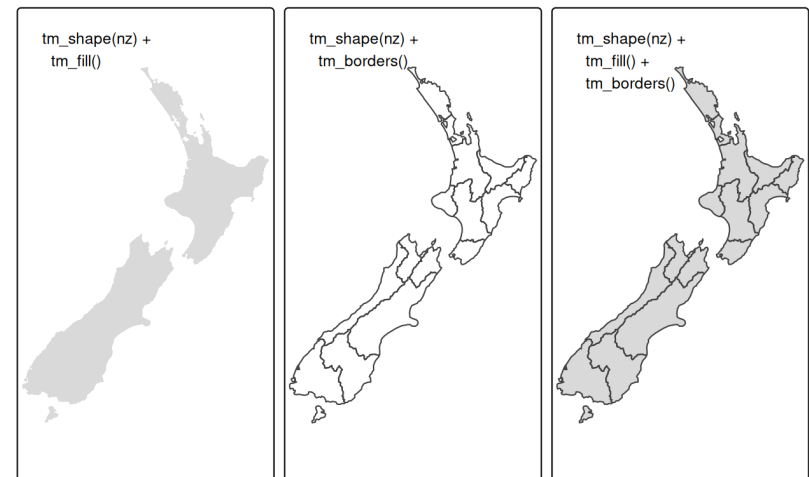
Example

```
library(sf)
library(terra)
library(dplyr)
library(spData)
library(spDataLarge)

library(tmap) # for static and interactive maps
library(leaflet) # for interactive maps
library(ggplot2) # tidyverse data visualization package

nz_elev = rast(system.file("raster/nz_elev.tif",
                           package = "spDataLarge"))

# Add fill layer to nz shape
tm_shape(nz) + tm_fill()
# Add border layer to nz shape
tm_shape(nz) + tm_borders()
# Add fill and border layers to nz shape
tm_shape(nz) + tm_fill() + tm_borders()
```



Data Representation

Rate vs Count

- **Count (Absolute):** The raw number of occurrences within a boundary (e.g., total number of traffic accidents, total population).
- **Rate (Relative):** A value derived by dividing the count by a denominator, such as population or land area (e.g., accidents per 1,000 residents, cases per square mile).
- "Population Bias" Trap ... when mapping Counts at the geographic level:
 - Large or densely populated counties will almost always appear as "hotspots."
 - You end up mapping where people live rather than the intensity of the variable.
 - Example: Mapping total COVID-19 cases will always highlight Los Angeles and New York City, even if the risk of infection is higher in a small rural town.

Feature	Use Count When...	Use Rate When...
Objective	You need to know the total magnitude or volume.	You need to compare the intensity or risk across regions.
Resource Allocation	Determining how many total vaccines to ship to a county.	Determining which county has the highest infection risk.
Visual Encoding	Best used with Proportional Symbols (bubbles).	Best used with Choropleth Maps (shaded regions).

"Never shade a county map based on raw counts." >> To normalize data for fair comparison, always convert counts to rates (normalization) using a common denominator ($\text{Rate} = \text{Count} / \text{Population} \times 100,000$) or density ($\text{Density} = \text{Count} / \text{Area}$).

```

library(sf)
library(dplyr)
library(ggplot2)
library(patchwork)
library(viridis)

# Create simple square geometries to represent counties in a grid
polys <- st_sfc(
  st_polygon(list(matrix(c(0,0, 1,0, 1,1, 0,1, 0,0), ncol=2, byrow=TRUE))), # Bottom Left
  st_polygon(list(matrix(c(1,0, 2,0, 2,1, 1,1, 1,0), ncol=2, byrow=TRUE))), # Bottom Right
  st_polygon(list(matrix(c(0,1, 2,1, 2,2, 0,2, 0,1), ncol=2, byrow=TRUE))) # Top (Large)
)

# Convert to sf object and add the table data
df <- data.frame(
  name = c("Rural County", "Suburban County", "Urban County"),
  total_incidents = c(50, 100, 500),
  population = c(5000, 50000, 1000000),
  incident_rate = c(10.0, 2.0, 0.5) # Incidents per 1,000
)

# Join geometry to data
comparison_data <- st_sf(df, geometry = polys)

# Add centroid coordinates for labeling
comparison_data <- comparison_data %>%
  mutate(
    centroid = st_centroid(geometry),
    lon = st_coordinates(centroid)[,1],
    lat = st_coordinates(centroid)[,2])

```

```
# 2. CREATE MAP 1: WRONGFUL (COUNT) -----
```

```
map_count <- ggplot(comparison_data) +  
  # Shade by Total Incidents  
  
  geom_sf(aes(fill = total_incidents), color = "black", size = 0.5) +  
  # Color scale  
  scale_fill_viridis_c(option = "rocket", begin = 0.3, end = 1) +  
  
  geom_text(aes(x = lon, y = lat, label = name),  
            fontface = "bold", nudge_y = 0.2, check_overlap = TRUE) +  
  geom_text(aes(x = lon, y = lat, label = paste("Count:", total_incidents)),  
            size = 3.5, nudge_y = -0.1) +  
  geom_text(aes(x = lon, y = lat, label = paste("Pop:", format(population, big.mark=","))),  
            size = 3.5, nudge_y = -0.3) +  
  
  theme_void() +  
  labs(  
    title = "Map 1: Wrongful Showing (Raw Count)",  
    subtitle = "Darker colors focus only on magnitude, ignoring population size.",  
    fill = "# of\nIncidents"  
  ) +  
  
  theme(  
    plot.title = element_text(color = "red", face = "bold", size = 14),  
    legend.position = "right",  
    plot.margin = margin(10,10,10,10)  
  )
```

```
# 3. CREATE MAP 2: CORRECT (RATE) -----
map_rate <- ggplot(comparison_data) +
  geom_sf(aes(fill = incident_rate), color = "black", size = 0.5) +
  scale_fill_viridis_c(option = "mako", direction = -1) +
  geom_text(aes(x = lon, y = lat, label = name),
            fontface = "bold", nudge_y = 0.2, check_overlap = TRUE) +
  geom_text(aes(x = lon, y = lat, label = paste("Rate:", incident_rate)),
            size = 3.5, nudge_y = -0.1, color = "white") +
  theme_void() +
  labs(
    title = "Map 2: Correct Showing (Normalized Rate)",
    subtitle = "Darker colors focus on intensity, revealing the true hotspot.",
    fill = "Rate per\n1,000 Pop."
  ) +
  theme(
    plot.title = element_text(color = "green4", face = "bold", size = 14),
    legend.position = "right",
    plot.margin = margin(10,10,10,10)
  )
)
```

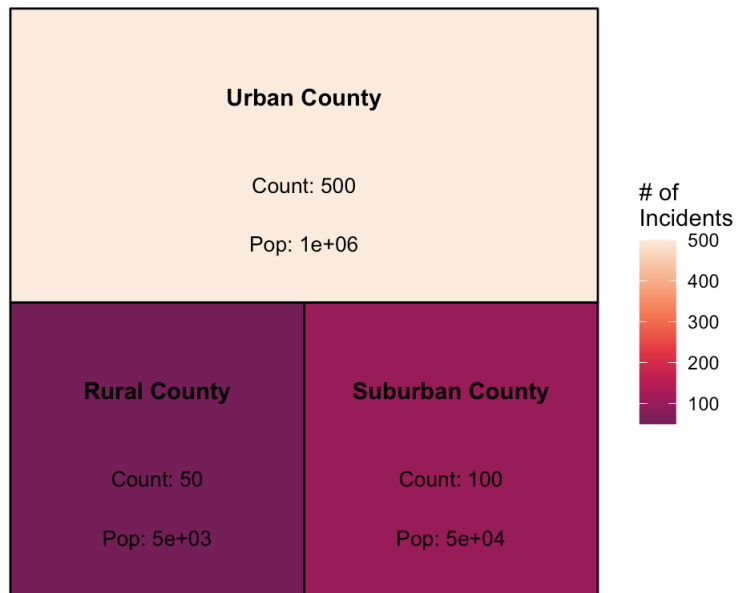
4. DISPLAY COMPARISON USING PATCHWORK -----

```
comparison_plot <- map_count / map_rate # Stack them vertically
comparison_plot <- map_count + map_rate # Stack them vertically

comparison_plot
```

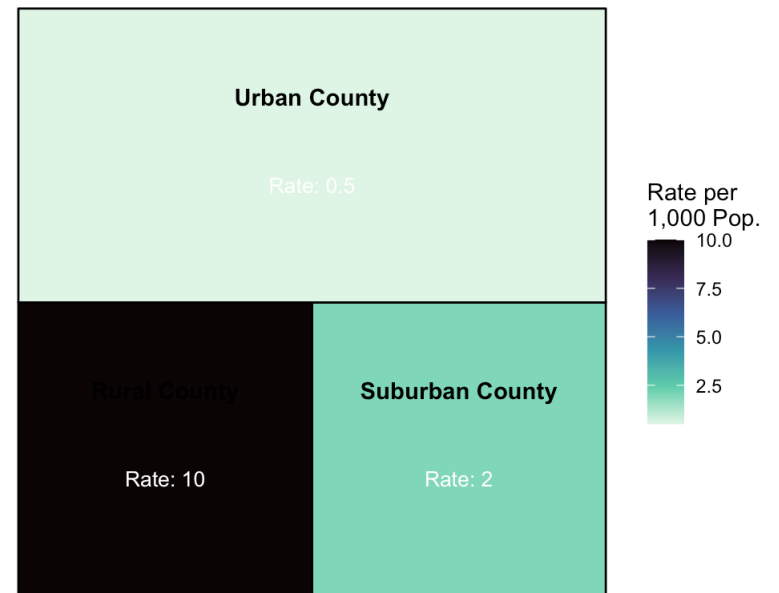
Map 1: Wrongful Showing (Raw Count)

Darker colors focus only on magnitude, ignoring population size.



Map 2: Correct Showing (Normalized Rate)

Darker colors focus on intensity, revealing the true hotspot.



Data Representation

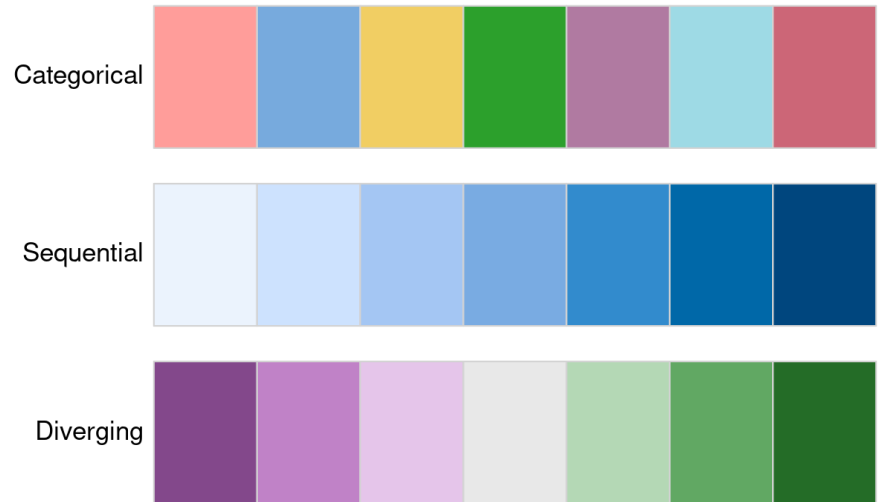
Color

There are two important principles for consideration when working with colors:

perceptibility and **accessibility**.

[R color cheatsheet](#)

[ColorBrewer: Color Advice for Maps](#)



- **Categorical palettes**

- Consist of easily distinguishable colors and are most appropriate for categorical data without any particular order such as state names or land cover classes.
- Colors should be intuitive: rivers should be blue, for example, and pastures green.
- Avoid too many categories: maps with large legends and many colors can be uninterpretable.

- **Sequential palettes**

- Follow a gradient, for example from light to dark colors (light colors often tend to represent lower values), and are appropriate for continuous (numeric) variables.
- Sequential palettes can be single (greens goes from light to dark green, for example) or multi-color/hue.

- **Diverging palettes**

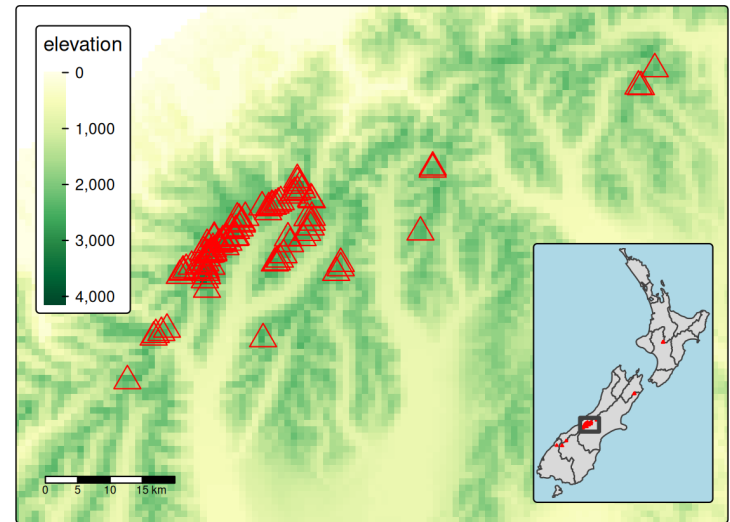
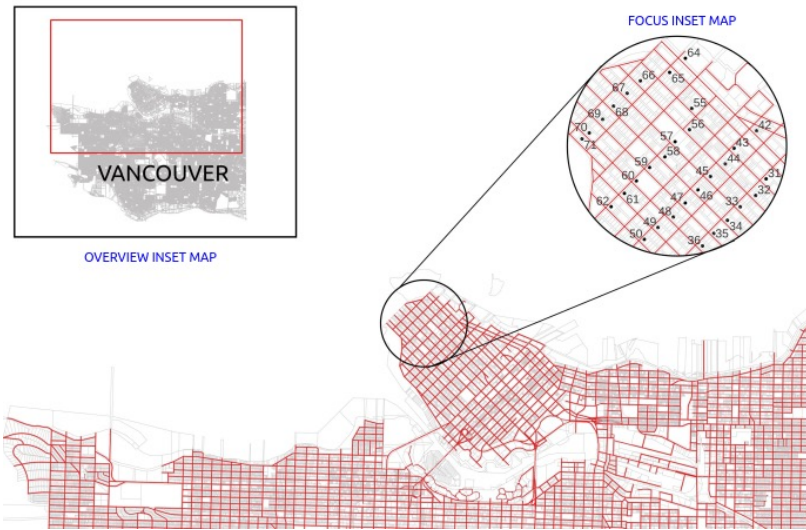
- Typically range between three distinct colors (purple-white-green) and are usually created by joining two single-color sequential palettes with the darker colors at each end.
- Their main purpose is to visualize the difference from an important reference point, e.g., a certain temperature, the median household income or the mean probability for a drought event.

Legend

- After we decided on our visual variable and its properties, we should move our attention toward the related map legend style.
- Using the **tm_legend()** function, we may change its title, position, orientation, or even disable it.
- The most important argument in this function is **title**, which sets the title of the associated legend.
- In general, a map legend title should provide two pieces of information:
 - what the legend represents
 - what the units are of the presented variable.
 - Check the scales ... **breaks** control how the values are represented on the map and in the legend, and they largely depend on the selected visual variable.
 - The default setting uses 'pretty' breaks
 - But **breaks** allows you to manually set the breaks
 - **n** sets the number of bins into which numeric variables are categorized
 - **values** defines the color scheme, for example, **BuGn**

Context Inset Maps

- An inset map is a smaller map rendered within or next to the main map.
- It could serve many different purposes, including providing a context or bringing some non-contiguous regions closer to ease their comparison.

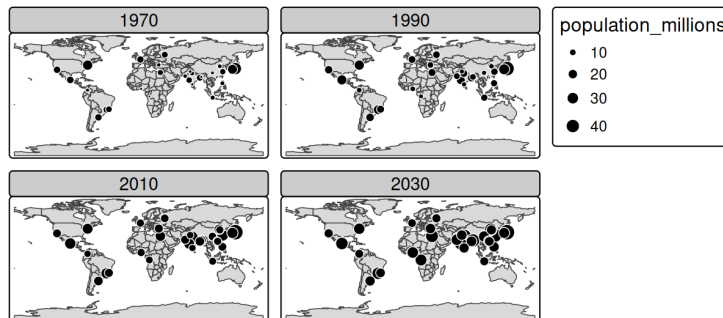


Faceted Maps

- Faceted maps, also referred to as 'small multiples', are composed of many maps arranged side-by-side, and sometimes stacked vertically.
- Facets enable the visualization of how spatial relationships change with respect to another variable, such as time.

Example

```
urb_1970_2030 = urban_agglomerations |>  
  filter(year %in% c(1970, 1990, 2010, 2030))  
  
tm_shape(world) +  
  tm_polygons() +  
  tm_shape(urb_1970_2030) +  
  tm_symbols(fill = "black", col = "white", size = "population_millions") +  
  tm_facets_wrap(by = "year", nrow = 2)
```



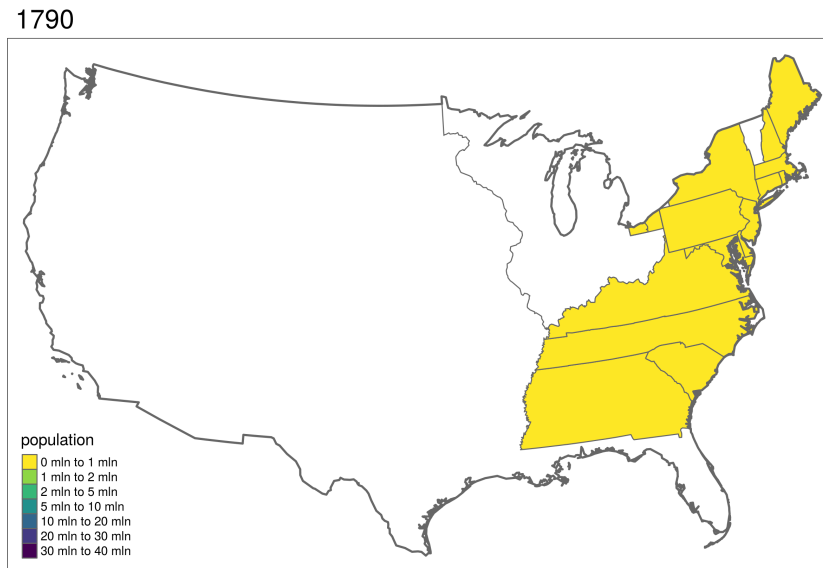
Animated maps

- Faceted maps can show how spatial distributions of variables change (e.g., over time), but the approach has disadvantages.
- Facets become tiny when there are many of them.
- Animated maps solve these issues. Although they depend on digital publication, this is becoming less of an issue as more and more content moves online.

Example

```
urb_anim = tm_shape(world) + tm_polygons() +  
  tm_shape(urban_agglomerations) + tm_symbols(size = "population_millions") +  
  tm_facets_wrap(by = "year", nrow = 1, ncol = 1, free.coords = FALSE)
```

```
tmap_animation(urb_anim, filename = "urb_anim.gif", delay = 25)
```



Interactive maps

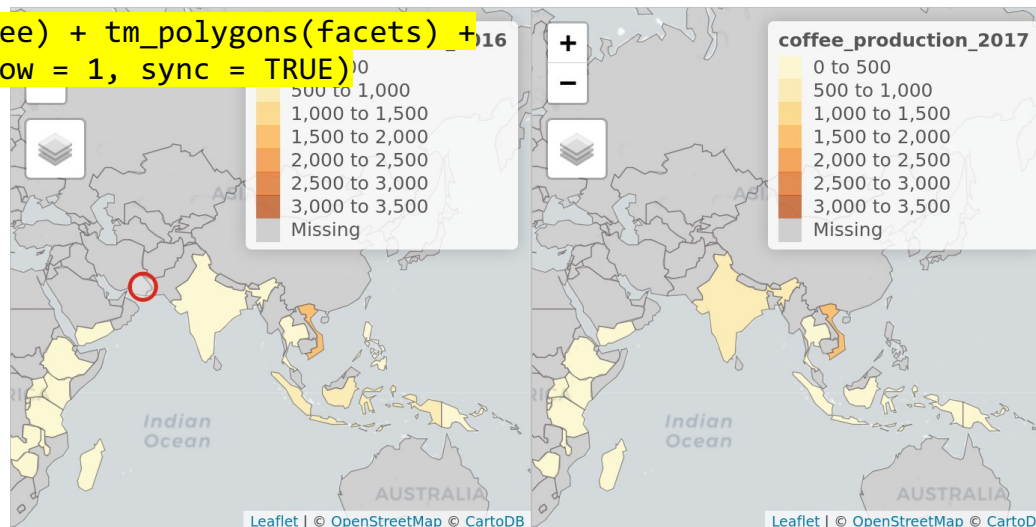
- While static and animated maps can enliven geographic datasets, interactive maps can take them to a new level.
- Interactivity can take many forms, the most common and useful of which is the ability to pan around and zoom into any part of a geographic dataset overlaid on a 'web map' to show context.
- Less advanced interactivity levels include pop-ups which appear when you click on different features, a kind of interactive label

Example

```
tmap_mode("view")
map_nz
map_nz + tm_basemap(server = "OpenTopoMap")
```

```
world_coffee = left_join(world, coffee_data, by = "name_long")
facets = c("coffee_production_2016", "coffee_production_2017")
```

```
tm_shape(world_coffee) + tm_polygons(facets) +  
  tm_facets_wrap(nrow = 1, sync = TRUE)
```



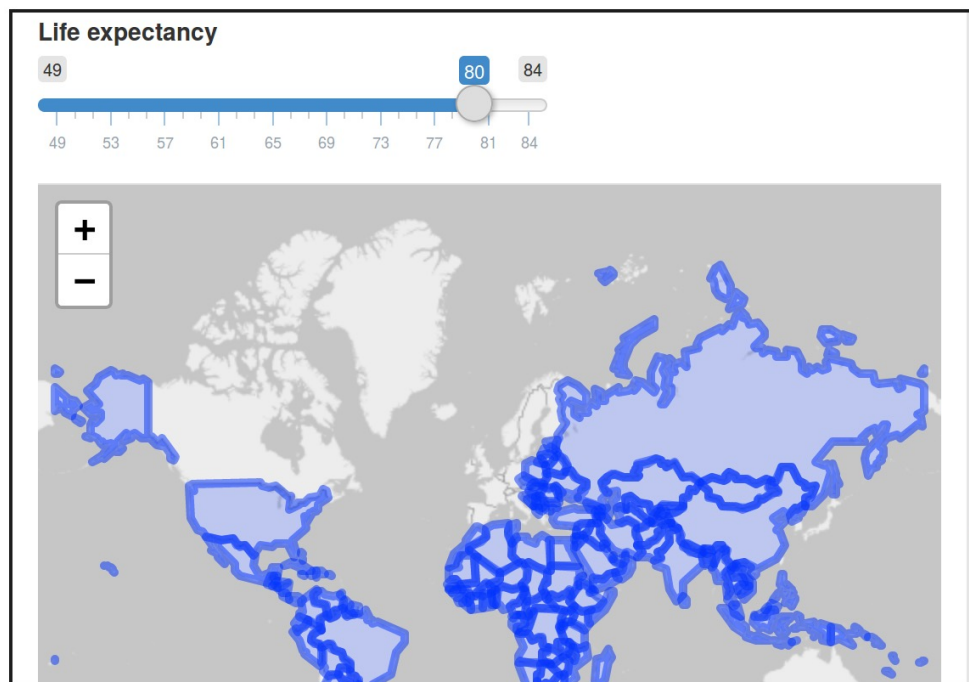
Mapping applications

- While interactive web maps (e.g., leaflet) allow for zooming and panning, mapping applications provide a higher level of interactivity and scalability by using server-side frameworks.
- Limitations of basic web maps:
 - **Fixed interface:** The UI is static and cannot be easily customized by the user.
 - **Data scaling:** Basic web maps struggle with very large datasets as all content is typically pre-loaded.
 - **Low complexity:** Difficult to integrate complex dashboards, real-time graphs, or multi-variable relationships.
- The Power of Shiny (for R Users):
 - **Rapid development:** Convert R code directly into interactive web apps without needing deep HTML/JS knowledge.
 - **Integration:** Seamlessly embed maps using functions like `leaflet::renderLeaflet()`
 - **Reactive logic:** Maps can update instantly based on user input, filters, or external data changes.
- **Enterprise-Level Framework:**
 - For high-traffic or massive data needs, R can interface with or be replaced by dedicated map servers:
 - **GeoServer / MapServer:** Mature, scalable servers for thousands of daily users.
 - **GeoDjango:** Python-based framework for complex geospatial web development.

```

library(shiny)    # for shiny apps
library(leaflet)  # renderLeaflet function
library(spData)   # loads the world dataset
ui = fluidPage(
  sliderInput(inputId = "life", "Life expectancy", 49, 84, value = 80),
  leafletOutput(outputId = "map")
)
server = function(input, output) {
  output$map = renderLeaflet({
    leaflet() |>
      # addProviderTiles("OpenStreetMap.BlackAndWhite") |>
      addPolygons(data = world[world$lifeExp < input$life, ]))
  }
shinyApp(ui, server)

```



More References

- Cartographic Design Process
<https://colorado.pressbooks.pub/makingmaps/chapter/cartographic-design-process/>
- Crime Mapping and Spatial Data Analysis using R
Chapter 6 **Basics of cartographic design: elements of a map**
https://maczokni.github.io/crime_mapping/basics-of-cartographic-design-elements-of-a-map.html

Next

- Getting started with statistical learning
 - **Exploratory** spatial data analysis
<https://cran.r-project.org/web/packages/geostan/vignettes/measuring-sa.html>
 - Measures of **spatial autocorrelation**
<https://r-spatial.org/book/15-Measures.html>
 - Point **pattern** analysis
<https://r-spatial.org/book/11-PointPattern.html>
 - Spatial **interpolation**
<https://r-spatial.org/book/12-Interpolation.html>

Notes

- Meet a new format: geoJSON
 - GeoJSON is an open standard format designed for representing simple geographical features, along with their non-spatial attributes.
 - It is based on JSON, the JavaScript Object Notation.
 - It is a format for encoding a variety of geographic data structures and is the most common format for geographical representation in the web.
 - Unlike ESRI shapefiles, with GeoJSON data everything is stored in a single file.